

Introduction to Java

Environment, IDEs, and Getting Started

Ahmed Elshiekh

February 17, 2026

Outline

- 1 Setting Up Java Environment
- 2 Java IDEs
- 3 Your First Java Program
- 4 Java Project Structure
- 5 Essential Java Concepts
- 6 Best Practices

Setting Up Java Environment

① Download and Install JDK

- Oracle JDK
- Choose appropriate version for your OS

② Set Environment Variables

- JAVA_HOME - Points to JDK installation directory
- PATH - Add \$JAVA_HOME/bin

③ Verify Installation

- `java -version`
- `javac -version`

Popular Java IDEs

IntelliJ IDEA

Most popular, powerful features, excellent for large projects

Eclipse

Free, open-source, extensive plugin ecosystem

NetBeans

Apache project, good for beginners

Visual Studio Code

Lightweight, great with Java extensions

Key Features:

- Smart code completion
- Powerful refactoring tools
- Integrated debugger
- Built-in version control

Editions:

- **Community Edition**
Free and open-source
- **Ultimate Edition**
Paid, advanced features

Advantages:

- Free and open-source
- Large community support
- Highly customizable
- Multi-language support

Built-in Support:

- Git version control
- Regular updates
- Long-term support

Best for:

- Enterprise Java development

Hello World Program

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, World!");  
4     }  
5 }
```

Key Points:

- File name must match class name: HelloWorld.java
- main method is the entry point
- public static void main(String[] args) is the signature

Compiling and Running Java Programs

Step 1: Compile

```
1 $ javac HelloWorld.java
```

This creates HelloWorld.class (bytecode file)

Step 2: Run

```
1 $ java HelloWorld  
2 Hello, World!
```

Note

IDEs handle compilation and execution automatically!

Typical Java Project Structure

Directory Layout:

`src/` Source code files (.java)

`bin/` Compiled class files (.class)

Core Java Concepts to Learn

Fundamentals:

- Variables and data types
- Operators
- Control flow (if-else, loops)
- Methods and parameters

Object-Oriented:

- Classes and objects
- Inheritance
- Polymorphism
- Encapsulation

Naming Conventions:

- Classes: PascalCase
- Methods: camelCase
- Constants: UPPER_CASE
- Packages: lowercase

Code Quality:

- Meaningful names
- Short, focused methods
- Single responsibility
- Proper exception handling

Development Practices:

- Write comments for complex logic
- Use version control (Git) from day one
- Write unit tests
- Follow DRY (Don't Repeat Yourself) principle